applied sciences

MDPI

Article

Enhancing Sustainable IoT Systems Through a Goal-Oriented Requirements Analysis Framework

José Alfonso Aguilar-Calderón [1,*], Carolina Tripp-Barba [1,*], Aníbal Zaldívar-Colado [1], Pedro Alfonso Aguilar-Calderón [2] and Pablo Alfonso Aguilar-Calderón [3]

1 Facultad de Informática Mazatlán, Universidad Autónoma de Sinaloa, Mazatlán 82017, Mexico; azaldivar@uas.edu.mx
2 Facultad de Ingeniería y Tecnología Mazatlán, Universidad Autónoma de Sinaloa, Mazatlán 82017, Mexico; pedro_a4@uas.edu.mx
3 Facultad de Derecho Mazatlán, Universidad Autónoma de Sinaloa, Mazatlán 82017, Mexico; pablo_aguilar@uas.edu.mx
* Correspondence: ja.aguilar@uas.edu.mx (J.A.A.-C.); ctripp@uas.edu.mx (C.T.-B.)

Abstract: The rapid expansion of the Internet of Things (IoT) has introduced significant challenges in requirements engineering (RE) due to the complexity of heterogeneous devices and dynamic user needs. Traditional RE methodologies often result in inefficient resource utilization and poor system performance. This research presents the goal-oriented requirements analysis (GORA) methodology, which optimizes requirement specification, resource allocation, and sustainability in IoT development. GORA addresses both functional and non-functional requirements, such as energy efficiency and security, while minimizing computational overhead and reducing resource wastage. The methodology integrates goal-oriented requirements analysis language i* and model-driven development (MDD) through a three-stage transformation process involving the i* RA model, UML class diagrams, and Python source code generation. A case study demonstrates how GORA improves system reliability, performance, and sustainability. Furthermore, an empirical evaluation was conducted in a simulated IoT environment, measuring key metrics such as energy consumption, latency, and code-generation error rate. A comparative analysis with existing RE and MDD approaches is also presented to highlight GORA's advantages in feature coverage, sustainability support, and automation level. This work underscores the need for structured IoT RE frameworks and positions GORA as a foundation for future research in sustainable IoT development.

Keywords: IoT; sustainable IoT systems; requirements analysis; internet of things; IoT software systems development; goal-oriented requirements analysis; GORE

check for updates

Academic Editor: Luis Javier Garcia Villalba

Received: 26 March 2025
Revised: 9 May 2025
Accepted: 20 May 2025
Published: 22 May 2025

Citation: Aguilar-Calderón, J.A.; Tripp-Barba, C.; Zaldívar-Colado, A.; Aguilar-Calderón, P.A.; Aguilar-Calderón, P.A. Enhancing Sustainable IoT Systems Through a Goal-Oriented Requirements Analysis Framework. *Appl. Sci.* **2025**, *15*, 5826. https://doi.org/10.3390/app15115826

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

The Internet of Things (IoT) has emerged as a transformative paradigm, significantly impacting diverse sectors, such as healthcare, smart cities, industrial automation, environmental monitoring, and intelligent transportation [1]. The integration of embedded devices, sensors, and ubiquitous communication networks has enabled the development of IoT solutions that facilitate real-time data exchange, automation, and decision making [2]. Currently, IoT finds applications in various domains, such as logistics [3], smart cities [4], healthcare systems (especially during the COVID-19 pandemic) [5,6], robotics [7], image recognition [8–10], and agriculture [11]. The impact of this versatile network spans across numerous sectors, paving the way for life-changing possibilities and innovations. However, despite the rapid adoption of IoT technologies, their development and sustainability

pose significant challenges, particularly in the requirements engineering (RE) phase. The inherent complexity of IoT systems, characterized by heterogeneous devices, real-time constraints, dynamic user needs, and resource limitations, requires a systematic, goal-driven approach to requirements analysis and system design.

The rapid evolution of IoT technologies has led to a proliferation of technical solutions, yet user support remains deficient. Unlike traditional software, IoT systems serve a diverse user base with varying needs, necessitating meticulous RE to address both functional (FR) and non-functional requirements (NFRs). RE, a subdiscipline of software engineering (SE), focuses on defining the objectives, functionalities, and constraints of the system [12]. As highlighted in [13], RE involves the analysis, specification, and validation of stakeholder needs to produce a comprehensive requirements document. Effective RE requires collaboration with customers to ensure that the final product satisfies their expectations. However, poor RE implementation often results in systems that do not meet user needs or remain unused.

The motivation for this research stems from the increasing demand for sustainable and efficient IoT systems that can operate under resource constraints while maintaining high performance. Traditional SE methodologies are often proven inadequate for the IoT development of IoT due to their inability to capture the intricate interactions between hardware, software, and user expectations [14]. Unlike conventional software systems, IoT solutions interact directly with the physical world through sensors and actuators, necessitating precise definitions of FRs and NFRs, such as energy efficiency, security, latency, and scalability. The absence of a well-defined RE framework tailored for IoT has led to inefficient development processes, excessive resource consumption, and frequent failures to meet end user expectations. Furthermore, existing RE methodologies struggle to address the multidimensional nature of the Internet of Things, resulting in poor requirement definition and high maintenance costs. A critical challenge is the need to reconcile diverse user expectations with system functionality through a goal-driven approach that ensures alignment between stakeholder needs and technical implementation. Furthermore, IoT nodes face inherent resource limitations, including constraints in processing power, memory, and battery life, which traditional SE approaches do not adequately address. Lastly, the environmental impact of IoT systems highlights the need to embed sustainability-focused design principles to promote long-term viability and resource-efficient development. Addressing these challenges requires a structured and comprehensive RE framework that optimizes requirements specifications while ensuring the implementation of an efficient and sustainable IoT system.

This research presents a systematic approach to requirements analysis in IoT software systems, addressing key challenges in RE. The proposed methodology aims to bridge three fundamental gaps in current IoT development practices: (1) the lack of comprehensive requirements analysis methodologies tailored for IoT environments, (2) insufficient tool support for requirement definition, and (3) the absence of a unified framework that integrates both hardware and software components. By addressing these gaps, this study contributes to the development of sustainable, efficient, and systematic IoT solutions that incorporate both technical and environmental considerations. Moreover, the proposed approach integrates sustainability concerns, such as energy efficiency and computational optimization, throughout the development lifecycle, from requirement specification to system implementation. This ensures that IoT systems are not only optimally designed, but also aligned with long-term sustainability objectives. The need for this research is reinforced by well-documented correlations between rigorous RE practices and the successful deployment of IoT systems. Empirical evidence in the software engineering literature consistently demonstrates that inadequate RE practices contribute to project failures, particularly in

complex distributed environments, such as IoT [15]. The consequences of poor requirements specification include (i) excessive rework cycles during development, (ii) inefficient resource allocation, and (iii) systems that ultimately fail to meet stakeholder needs, which is an especially critical issue given the IoT's direct interaction with the physical world.

The proposed goal-oriented requirements analysis (GORA) is a structured, sustainability-driven methodology that is tailored for the development of IoT software. GORA provides a systematic approach to identifying, analyzing, and managing IoT-specific requirements, with a focus on optimizing resource use and minimizing development waste. By incorporating goal decomposition, requirement traceability, and sustainability constraints, GORA facilitates the design of energy-efficient, computationally optimized, and adaptable IoT systems, contributing to the overall sustainability of the solution. The proposed methodology for developing sustainable IoT systems integrates goal-oriented requirements engineering (GORE) and model-driven development (MDD) through a three-stage transformation process. The first stage involves defining the i* requirement analysis (RA) model, capturing the objectives and dependencies of the system. In the second stage, model-to-model (M2M) transformation rules are applied using Query/View/Transformation (QVT) to convert the i* RA model into a UML class diagram, ensuring a structured and well-defined representation of system components. Finally, in the third stage, a model-to-text (M2T) transformation is executed using Acceleo to generate Python source code from the UML class diagram, which streamlines the implementation of the IoT system. To ground this work in a real-world context, consider a homeowner aiming to improve indoor air quality using an IoT system. The system must collect data from multiple air quality sensors, visualize readings, and trigger actions, such as enabling air purifiers. This scenario exemplifies the complex interplay between stakeholder goals, system requirements, and sustainability constraints, which is introduced in detail in Section 4.2. This user journey encapsulates the typical input (goal definitions, environmental context, etc.) and expected output (context-sensitive device actions, alerts, and reports), serving as a reference example throughout the manuscript.

This research contributes to the field of IoT systems engineering in four key areas. First, it introduces a methodological innovation by proposing a structured RE framework tailored to the specific challenges of IoT environments, including device heterogeneity and real-time operational constraints. Second, it advances resource optimization through a goal-oriented approach that integrates computational and energy efficiency as fundamental requirements throughout the development lifecycle. Third, it pioneers sustainability integration by presenting the first RE methodology that systematically incorporates sustainability constraints at the requirements specification stage of IoT systems. Fourth, it establishes a theoretical foundation by defining formal traceability mechanisms that link high-level objectives to implementation requirements, providing a structured basis for future research in IoT RE. Furthermore, this integrated framework improves the efficiency, reliability, and sustainability of the development of IoT systems. By emphasizing rigorous requirement traceability and resource-conscious design, this study positions GORA as a foundational methodology for sustainable IoT RE, fostering future advancements in IoT software engineering.

The organization of this article is as follows. Section 2 provides an overview of the essential RE concepts pertinent to the context of this article. In Section 3, the existing related work in this field is explored. Section 4 introduces the goal-oriented requirements analysis proposal for IoT software systems, as well as an example application. Section 5 presents an empirical evaluation of the proposed approach. A comparative analysis with existing approaches is presented in Section 6. Finally, in Section 7, we conclude the article and outline potential avenues for future research.

2. Fundamentals

In this section, essential concepts related to the context of this proposal will be explored, including the IoT, as well as the phases of RE, MDD, and the GORE language known as i* (iStar).

2.1. The Internet of Things

The IoT is a versatile paradigm that enables interconnected devices to collect and exchange information from the surrounding environment for a wide range of applications. These include notification systems that inform customers about supermarket discounts when they are near them, food ordering applications that facilitate meal planning and grocery shopping, automotive monitoring solutions that track fluid levels to ensure optimal vehicle performance, and wearable devices capable of real-time heart rate monitoring, among others. Unlike traditional computing and monitoring systems, IoT is characterized by small, frequent data transmissions and the capacity to support a vast number of connected devices, distinguishing it from conventional approaches. The rapid evolution of IoT technologies has led to their widespread adoption in various domains, making them an integral component of modern digital ecosystems [16].

The IoT disseminates data to both users and various systems or software. As an illustration, when it comes to users, the IoT can transmit health-related information through software that collects data from sensors responsible for monitoring vital signs. Additionally, in numerous instances, businesses or individuals have the capability to remotely control a device to perform tasks, like activating the garage lights or scheduling an office air conditioning system to start refreshing the environment at 9:00 am in anticipation of employees' arrival.

2.2. Requirements Engineering

The use of RE phases plays a fundamental role in influencing the success of development projects within most software factories. RE involves the systematic process of identifying, studying, detailing, and validating the functionalities that a piece of software should provide, as well as its operational constraints. Several approaches exist to delineate the stages of RE, as proposed in sources such as [17,18]. Naturally, the specific phases can vary depending on factors such as the application domain and the composition of the development team. However, some universally recognized phases typically emerge, including elicitation, analysis, specification, validation, and management. The key phases of requirements engineering are detailed below.

- Elicitation: Elicitation identifies the key issues and objectives for the software. Techniques such as questionnaires, brainstorming, prototyping, and conceptual modeling are used in goal-oriented languages for this purpose [18–20].
- Analysis: In the analysis phase, models or prototypes are created to meet the identified requirements, including understanding organizational goals and analyzing relevant information, rules, and stakeholder communication [20].
- Specification: Specification involves defining system behavior using techniques such as templates, scenarios, use cases, modeling, and natural language [21].
- Validation: The validation phase ensures that the software requirements accurately reflect, using reviews and traceability methods, the needs of the stakeholders [21,22].
- Management: Management identifies and addresses, using configuration management and version control techniques, the potential requirement changes during development [22].
- Non-functional Requirements: These requirements ensure system quality and include aspects like usability, privacy, availability, interoperability, and accessibility.

2.3. Goal-Oriented Requirements Analysis Modeling Language iStar 2.0

Goal-oriented requirements engineering (GORE) is a systematic and formalized approach to software requirements engineering that centers around the identification, analysis, specification, and management of stakeholder goals and objectives. GORE methods have been prominent in software engineering, finding applications in diverse domains such as requirements engineering, organizational modeling, adaptive system modeling, and software architecture representation, .

In GORE, the main focus is on understanding the desired outcomes, needs, and intentions of stakeholders and translating these high-level goals into well-defined concrete requirements for a software system [20]. This process involves specialized modeling languages and techniques to represent and analyze dependencies, conflicts, and trade-offs among different goals, as well as the means and constraints to achieve them. GORE ensures that software systems align effectively with stakeholders' objectives while providing a structured and traceable foundation for the entire software development lifecycle.

The i* framework is primarily a requirements modeling framework that is used for both REs and RAs in GORE. In the analysis phase of IoT requirements, the i* framework serves as an RA framework, specifically within GORE. It helps model and analyze stakeholder goals, dependencies, and constraints, making it highly relevant for structuring IoT requirements analysis. The i* framework provides a means to represent and analyze relationships (represented as (), dependencies (represented as help hurt), and constraints between various stakeholders, their goals, tasks, and the resources needed to achieve those goals.

A refined version of i*, known as i* 2.0 (iStar 2.0), enhances the foundational principles of goal-oriented methodologies. The core language iStar 2.0 refines the fundamental concepts of i*, establishing a cohesive and transparent set of core principles. This foundation serves as a platform for advancing future developments and creating educational materials focused on goal-oriented methodologies.

The formal delineation of iStar 2.0 encompasses the following key components:

- Intentional Elements: Goals (ellipse), soft goals (eight-shaped), tasks (hexagon), and resources (rectangle) that represent the intentions of stakeholders.
- Actors (circle): Entities (e.g., individuals, organizations, or systems) that pursue goals and perform tasks.
- Actor Links: Relationships between actors, such as dependencies or collaborations.
- Intentional Elements Links: Connections that capture goal refinement, task decomposition, or means-end relationships.
- Model Views: Different perspectives facilitate stakeholder analysis and decision making, such as the Strategic Dependency (SD) view () and the Strategic Rationale (SR) view (depicted as a dashed circle).

2.4. Relevance of I* in IoT Requirements Analysis

The i* framework offers several advantages for the analysis of IoT requirements. First, it effectively captures stakeholder intentions by systematically modeling the diverse objectives of users, developers, and service providers involved in IoT systems. Second, it handles the inherent complexity of IoT requirements, including functional and non-functional requirements, like energy efficiency, security, and real-time constraints, thereby facilitating their analysis and balance. Third, the framework supports sustainability considerations by incorporating sustainability goals and constraints during the initial requirement analysis, thus helping to optimize resource consumption. Finally, i* models the interdependencies among IoT components, providing a holistic view of the system's needs and ensuring proper representation of actor dependencies.

Given the inherent complexities associated with the analysis of IoT requirements, the adoption of a comprehensive and well-defined RE framework specifically tailored to the IoT domain is imperative. The integration of GORE and the i* framework offers a structured methodological approach that significantly enhances the efficiency, reliability, and sustainability of IoT systems. By organizing the requirements analysis process around goal decomposition, formal requirement traceability, and sustainability constraints, the i* framework facilitates the development of IoT systems that emphasize energy efficiency, computational optimization, and adaptability. For a more detailed examination of iStar 2.0, refer to [23]. This study contributes to the establishment of a standardized RE framework for IoT systems, ensuring that IoT applications are not only functional and scalable, but that they are also resource-efficient and environmentally sustainable.

3. Background

The rapid proliferation of IoT technologies has ushered in a transformative era that is changing the way we interact with the world around us. As IoT ecosystems continue to expand, they offer unprecedented opportunities for innovation and connectivity. However, these opportunities come hand in hand with intricate challenges, particularly in software system development. This section shows related research focused on the role that goal-oriented requirements engineering plays as an enabler of effective, efficient, and adaptive IoT software systems. Through this exploration, we will lay the foundation for understanding the imperative need for an approach that can seamlessly align the goals of IoT stakeholders with the intricate demands of software systems development within this dynamic ecosystem.

The authors in [24] suggest a novel methodology that seamlessly blends social and goal-driven principles into system design, simulation, and implementation. This approach strongly emphasizes monitoring system quality and ensuring compliance throughout the development lifecycle. In this innovation, user objectives take center stage from the outset, guiding the selection of system architectures and design alternatives. Additionally, user goals remain a focal point, even during dynamic adaptations at run time. Remarkably, this methodology achieves its goals without redundancies or loss of valuable information. It accomplishes this by generating functions derived from goal and feature models, which are seamlessly integrated into SysML models and implementation processes. In particular, this approach leverages existing tools to minimize coding errors and reduce the effort required for these adjustments.

The research in [25] presents findings on addressing environmental uncertainty within the engineering of requirements based on goals. The motivation behind this research stems from the limitation of existing goal-based approaches, which lack built-in mechanisms for handling unexpected adaptations. The authors of that study discussed the integration of RELAX concepts into SysMLKaos. RELAX, a requirements engineering language tailored for Dynamically Adaptive Systems, introduces explicit constructs designed to manage the inherent uncertainty characteristic of such systems. In contrast, SysMLKaos, a goal-based requirements engineering methodology, uniquely considers non-functional requirements on par with functional requirements, effectively modeling the influence of non-functional requirements on functional requirements.

The growing complexity of cyberphysical systems (CPSs) poses significant design, development, and operation challenges. CPSs involve numerous heterogeneous components, which are often designed by different engineering organizations, making it a daunting task to manage, trace, and verify their properties, requirements, and constraints using traditional methods. Given these challenges, the authors of [26] introduced an integrated approach that formally defines system requirements and automates their verification through simula-

tion. This approach relies on the formal requirement model language and the Modelica language. To illustrate its effectiveness, the approach is demonstrated through two case studies that involve a trailing edge high-lift system and a Heating, Ventilation, and Air Conditioning system.

In [27], the authors discussed the fundamental challenges of transitioning from conventional manufacturing to a CPPS. This transformation represents a significant undertaking for requirements engineers tasked with identifying, specifying, and analyzing the implications of converting numerous assets into a network of interconnected devices, data sources, and human participants. Their paper introduced the e-CORE approach, which offers a systematic, analytical, and traceable method for requirements engineering, and it is grounded in a 'capability-oriented' paradigm with a strong focus on early requirements. To showcase its effectiveness, the approach was applied to two real-world use cases drawn from a large-scale application in the automotive industry with a specific focus on the requirements of Fiat Chrysler Automobiles. The authors presented a structured approach to requirements engineering tailored for CPPS applications.

In [28], a GORE approach was introduced to define needs and requirements within the Ambient Assisted Living (AAL) domain. Subsequent to the requirements mapping, the authors also put forth an initial architectural concept for a home care system called "e-Ward". This system was designed to provide assistance to patients in their home environment, simulating the environment of a hospital room. This approach is envisioned as a foundational step toward the development of a modular AAL framework capable of addressing all identified needs. The proposed architecture represents progress in this direction as it models a home care system that replicates a hospital room within a patient's home environment.

The authors in [29] presented a domain-specific modeling language tailored for the IoT as it was developed with UML (Unified Modeling Language) and considered IoT systems, as well as technical and nontechnical end users. Standardized and widely accepted usability metrics were used to validate its suitability, demonstrating that the language is user-friendly in real-life smart home IoT environments. The experimental results yielded auspicious results, with the language receiving consistently high usability scores from technical and non-technical users. Although these results are encouraging, there is a reason for further testing of the language in the context of more intricate IoT systems that require advanced expertise. However, this goes beyond the scope of this article. Furthermore, because of its compatibility with UML, advanced UML notations could be harnessed to enhance the expressiveness of the language. Alternatively, a subset of these notations could be customized to better align with the specific requirements of IoT applications.

In [30], a systematic mapping study was introduced to identify and comprehend crucial requirements for architectural modeling within IoT applications. The primary findings underscored that current methodologies have overlooked specific requirements and that only a limited number of software architecture approaches consider specialized considerations when modeling IoT applications.

Typically, a separation exists between architectural efforts and systems engineering (SE), resulting in a lack of traceability from system requirements and design to the overall architecture. However, Industrial Internet of Things (IIoT) systems bring together and seamlessly integrate systems and software engineering with enterprise systems, business processes, and analytics, providing valuable support to decision makers. In [31], the authors investigated an emerging international standard, the Unified Architecture Framework (UAF); they explored its potential application in capturing IIoT architectures and aiding systems integrators in developing interoperable systems while ensuring traceability across various viewpoints and the initial requirements. The analysis included mapping the

Industrial Internet Reference Architecture (IIRA) to the UAF and illustrating its practical application through a real-world case study, demonstrating how the UAF can effectively model IIoT architectures.

In [32], an IoT-based smart home system proposal, which outlines the system architecture based on an Internet of Things layered framework, was introduced. Furthermore, their paper provided a comprehensive design of the system's gateway, covering hardware and software aspects in detail. It delves into the interface circuits for components such as the Zigbee module, the 3G module, and the video module concerning the S3C2440A microprocessor. In addition, their paper described the software implementation that runs on an embedded Linux operating system. The subsequent focus will be on optimizing the system's power consumption.

The authors in [33] comprehensively overviewed the contemporary software engineering approach to systematically developing IoT systems and applications. The author defined the essential principles and conceptual foundations that underpin the design and creation of IoT systems and applications. Furthermore, their paper outlined a software engineering methodology around these foundational concepts, emphasizing the importance of harnessing agent-oriented software engineering and software engineering techniques tailored for self-adaptive and self-organizing systems.

The continuous expansion of the IoT has created unprecedented opportunities for technological advancement and interconnectivity. However, ensuring the sustainability of IoT systems remains a critical challenge due to their inherent complexity, resource constraints, and dynamic operational environments. This section has provided a comprehensive review of research contributions that highlight the role of GORE as a fundamental approach to improving the efficiency, adaptability, and sustainability of IoT software systems. The reviewed studies underscore the importance of structured methodologies in addressing key sustainability concerns in the development of IoT. Research has demonstrated the benefits of integrating goal-driven principles into system design, monitoring, and compliance [24], as well as the need for uncertainty management in dynamically adaptive IoT environments [25]. Furthermore, formal approaches to requirements specification and automated verification [26] provide a foundation for developing reliable and resource-efficient IoT applications. The transition to CPPSs [27] further illustrates the need for sustainable and traceable requirements engineering frameworks, ensuring long-term viability in industrial settings. In addition, advances in domain-specific modeling languages for IoT [29], systematic architectural analyses [30], and the application of unified frameworks in IIoT [31] emphasize the role of structured methodologies in achieving energy efficiency, computational optimization, and environmental responsibility. The contributions in smart home systems [32] and agent-based approaches to self-adaptive IoT applications [33] further reinforce the need for sustainability-oriented design paradigms.

In the context of engineering heterogeneous IoT systems, interoperability, policy enforcement, and standard-compliant specification mechanisms are essential prerequisites. Recent studies have emphasized the need for system-understandable models to abstract device capabilities, constraints, and interactions in a uniform manner. For example, Xu et al. [14] highlighted the complexity of policy enforcement in dynamic IoT environments and proposed architectural patterns to ensure compliance across heterogeneous platforms. Similarly, Bernabé et al. [34] advocated for a semantic abstraction layer that enables interoperability by formally describing interactions between devices within smart spaces. These perspectives align with the foundational assumptions of the GORA methodology, which is based on formal goal models and model transformations to ensure consistency and adaptability between system components. Moreover, adherence to emerging IoT standards, such as the Web of Things (WoT) and oneM2M, further enhances interoperability and reusability. These stan-

dards provide standardized vocabularies, communication interfaces, and semantic descriptors that can be leveraged to bridge the gap between high-level requirements and device-level capabilities. By embedding sustainability goals into a model-driven pipeline that is compatible with such standards, GORA supports scalable integration and policy-compliant behavior in multi-device ecosystems.

This synthesis of existing research highlights the imperative for a robust, goal-oriented RE framework that prioritizes sustainability in the development of IoT systems. By aligning stakeholder objectives with considerations of energy efficiency, computational optimization, and long-term environmental impact, this approach ensures that IoT applications are not only functionally effective and scalable, but also ecologically responsible and resilient. The findings presented in this section establish a foundational basis for exploring structured methodologies that integrate sustainability as a core principle in IoT requirements engineering.

4. Goal-Oriented Requirements Analysis Approach for IoT Software Systems

To effectively address the complexities of the application domain, IoT systems require an appropriate modeling language and a structured analysis method that guide business analysts and process designers. In this context, a requirements analysis (RA) approach using the i* 2.0 modeling language within a model-driven framework is proposed. The components have been adapted to align with the specific terminology of IoT systems. To illustrate the approach, an application example is presented that focuses on the integration of IoT devices for air quality monitoring. By detailing this methodology, this research ensures transparency and reproducibility, facilitating its adoption in similar contexts.

To further enhance the sustainability of IoT systems, the proposed framework integrates goal-oriented requirements analysis with MDD, providing a structured approach to linking user goals with the corresponding system actions required to achieve them. This is achieved by utilizing (i) task elements from the i* modeling language to analyze the impact of each device action and (ii) goals and sub-goals (quality attributes) to ensure a user-centric orientation in the requirements analysis. As a result, the system dynamically selects and executes device actions based on the current environmental state and the intended goals, enabling context-aware decision making.

Designed for non-technical end users, the framework leverages the i* modeling language, which offers straightforward semantics and intuitive reasoning mechanisms, ensuring ease of adoption with a short learning curve. This approach provides a comprehensive overview of the IoT system while maintaining a strong user focus. End users can define and express their desired states in the smart environment through various interaction modes, including web-based applications, voice commands, or mobile applications, all of which are incorporated into the modeling process. Furthermore, the framework is inherently adaptable, allowing users to customize their goals through the user interface without requiring technical expertise or direct manipulation of the system logic.

The proposed methodology for developing sustainable IoT systems integrates GORE and MDD through a three-stage transformation process. The first stage involves defining the i* RA model, capturing the objectives and dependencies of the system. In the second stage, model-to-model (M2M) transformation rules are applied using Query/View/Transformation (QVT) to convert the i* RA model into a UML class diagram, ensuring a structured representation of system components. Finally, in the third stage, a model-to-text (M2T) transformation is performed using Acceleo to generate Python source code from the UML class diagram, facilitating the implementation of the IoT system. The following sections provide a detailed explanation of this transformation process.

4.1. Goal-Oriented Requirements Analysis Model in a Nutshell

To facilitate goal-oriented requirements analysis for IoT systems, the framework must possess the ability to establish a connection between the users' goals and the set of actions required to achieve those goals. To achieve this, our solution uses (i) task elements from i* to offer insights into the effects of each device action and (ii) goals and sub-goals (quality) to provide a user orientation analysis. Consequently, various device actions are executed for the same goal based on the current state of the devices in the environment and the goal to achieve, resulting in a context-aware system. As the proposal is tailored explicitly for non-technical end users, the i* language provides a simple semantics and reasoning approach with a short learning curve. Hence, a big picture regarding the IoT system is provided considering the user; the IoT system allows end users to express the desired state of their smart environment through a web-based application, voice interaction, or a mobile application (these are considered for modeling). The approach can be easily customized to meet new user requirements, enabling end users to add goals through the user interface without dealing with underlying technical complexities.

4.1.1. Adaptation Explanation

Technically, considering the adaptation of the i* language for the IoT requirements analysis, the representation of each modeling element in the i* 2.0 language for use in the requirements analysis for IoT systems is as follows:

- Actor: In numerous goal-oriented methodologies, actors play a crucial role by having intentions and goals. This introduces a seamless and powerful approach for scrutinizing, formulating, and executing various software solutions. Actors possess characteristics such as reactivity, proactivity, and social ability, making them practical for representing, analyzing, and designing software solutions and various complex systems that involve cooperation and co-creation of value. In this concept, actors can be employed to symbolize the end user, a software system (whether web-based, mobile application, or desktop), and the IoT system. They are labeled as "IoT" for the IoT system, "U" for the end user, and "S" for the web-based, mobile, or desktop software.
- Goal: These are the objectives that actors aim to achieve. The symbol is the same as the i* 2.0 language.
- Quality: Represents quality attributes. The symbol is the same as the core i* language. It can be used for quality issues in IoT system ("IoT") and user ("U") actors, as well as software attributes for the actor "S".
- Task: The actors perform actions to achieve their goals, acting as mechanisms to achieve the goals. In this proposal, tasks are essentially identified as the services provided or consumed by "things". They are usually associated with IoT system functionalities or software functions.
- Resource: This is an entity that is essential for an actor to perform a task. In the original i* language, the symbol was contemplated by simply adding a label with "T" since it represents a "thing". The "thing" is the essential element of the IoT system. This can be integrated with other "things" to construct an IoT system.

The adaptation previously introduced was implemented using the Eclipse Modeling Framework (EMF) through the use of a metamodel. EMF is a framework designed for modeling and code generation, facilitating the development of tools and applications based on a structured data model. Within this framework, a metamodel defines the structure and constraints of other models, serving as a formal specification that describes the permissible elements, their attributes, relationships, and constraints. In EMF, metamodels are typically defined using the Ecore metamodel, a subset of UML specifically designed to describe the structure of models within the EMF environment.

4.1.2. Procedure for Requirements Analysis Model Definition

Here are the steps to define the i* models proposed for the IoT requirements analysis. Initially, according to Section 4.1.1, it is crucial to identify the actors that will be part of the IoT ecosystem (context). This overview will offer a general understanding of the proposal, known as the big picture. Identification of intentional elements of the i* 2.0 language should be carried out. It is essential to note that intentional elements encompass intentions, such as tasks, resources, goals, or quality elements that actors desire. It is essential to establish a Strategic Dependency (SD) model for each actor within the context of the IoT system. The relationships between actors, such as the system and end users, can be modeled in a Strategic Rationale (SR) model. The hybrid view is used to integrate SD/SR perspectives to gain a comprehensive view of the system.

Consider an application example introduced in the following Section 4.2 to exemplify the proposal. The example is related to an IoT system for monitoring air quality.

4.2. Application Example: IoT System for Monitoring Air Quality

This subsection presents an example application of the proposed requirements analysis approach, utilizing the i* 2.0 modeling language, for IoT systems. The example is structured as a User Story, which serves as a general and informal description of a software feature from the perspective of the end user. The primary objective of a User Story is to convey the value that a software feature provides to its intended users. Although "customers" typically refer to external end users, in this context, they may also include internal stakeholders, such as colleagues or teams within an organization who rely on the system. User stories are formulated as concise statements in plain language that clearly define the expected results and benefits of the system's functionality.

4.2.1. User Story: Integration of IoT Devices for Air Quality Monitoring

Consider a scenario in which a homeowner, concerned about environmental health, seeks to implement an IoT-based system to monitor indoor air quality. Such a system would enable proactive measures to improve air quality and maintain a healthy living environment for the home. The key features of this system include the ability to receive customized alerts and seamless integration with other smart home devices, ensuring informed decision making and improved indoor air management.

4.2.2. User Story

- As a homeowner concerned about environmental health,
- I want to integrate IoT devices to monitor indoor air quality
- To obtain real-time information on pollutant levels and take proactive measures to improve indoor air quality.

4.2.3. Acceptance Criteria

- Air Quality Sensors: The application should be able to connect and recognize multiple IoT-based air quality sensors. Sensors must measure critical parameters, such as particulate matter, CO_2, humidity, and temperature.
- Real-Time Visualization: The application should provide an intuitive interface for real-time visualization of the data collected by the sensors. Air quality levels should be clearly represented.
- Customized Alerts: It should be able to set custom thresholds for each parameter and receive real-time alerts if the levels exceed these thresholds.

- Data History: The application must store and display a history of air quality data to enable long-term analysis. Users should be able to access detailed reports on trends and changes in air quality.
- Integration with Smart Systems: The application should be compatible with smart home systems, such as thermostats and air purifiers, to respond automatically to air quality data.

4.2.4. I* Requirements Analysis Model

The i* RA model detailed in Figure 1 illustrates the example presented in Section 4.2, where the i * hybrid view integrates SD/SR perspectives for the IoT environment in the application example. The requirements analysis model comprises three actors: The user, representing the end user in the User Story, which has the goal of "home air quality monitoring". The second actor is the Mobile App, the software through which the user can access real-time information on pollutant air levels in their house. The third actor, central to this example and further explained, is the IoT System, which serves as the core of the IoT of the User Story introduced in Section 4.2.1. According to Section 4.1.1, the i* 2.0 language element representing the IoT system is the actor element labeled "IoT". The goal of the system is "to obtain air information" and remember that a goal serves as an articulation of the intended outcome or accomplishment that a system or software is designed to achieve. These goals encapsulate the overarching objectives that stakeholders strive to attain by utilizing a system. In this case, the goal was defined according to the User Story from Section 4.2.1, which states that the end user in the "I Want" from the User Story wants to integrate IoT devices to monitor indoor air quality. This is the main goal; if this goal is satisfied, it will help to "improve air quality" since it will help the end user to make a decision support. To be fulfilled, it is necessary to satisfy two other goals: "establishing network connection" and "calculating sensor values". The goal "establish network connection" can be achieved in two forms (goals): "establish wired connection" or "establish wireless connection". This is represented by the connectors "OR Refinement". In this context, the goal "establish wired connection" allows for a reliable connection, as indicated by the quality element and the connection named contribution type "make" (expressing that "the source provides…"), which also affects "easy connection" since it is more complicated for the end user to establish a wired connection than a wireless one. In contrast, the goal of "establishing wireless connection" is the opposite. To fulfill the goal "calculate sensor values" (linked to the main goal by means of a refinement link, where the fulfillment of at least one child makes the parent fulfill), it is necessary to obtain the ID of the sensor (thing) through the service "get sensor identifier." This, in turn, requires fulfilling one of the two options: the service "read ID sensor" or "read another sensor identifier." These two services are connected through the "NeededBy" relationship, which links a service (task) with a thing (resource) and indicates that the actor IoT system needs the thing (resource) to execute the service (task). The things in this example are "humidity sensor", "CO_2 Sensor", and "temperature sensor," each of which have attributes such as "turn on", "off", and "restart" (which can be personalized as they are labels).

4.2.5. QVT for Model-to-Model Transformations

QVT is a model-to-model transformation language, with which it is possible to define a set of transformation rules to convert the i* RA model into a UML class diagram. The i* model will be transformed into a UML class structure that represents the classes, attributes, and relationships needed for the IoT air quality monitoring system.

To define a set of QVT transformation rules (model-to-model) for generating a UML class diagram from the i * RA model of an IoT system for air quality pollution monitoring,

it is necessary to break down the RA model into specific components that need to be transformed into class diagram elements. The transformation process must consider key aspects of sustainability, including energy efficiency, real-time data processing, and scalability. These rules will guide the process of converting the i* RA model to a class diagram, ensuring that the system adheres to these principles while focusing on achieving the goals of the IoT system. The QVT transformation rule will map the actors in the i* model to the classes in the UML model. Each actor in the i* model corresponds to a UML class that encapsulates the behavior and responsibilities of the actors in the IoT system. The goals in the i* RA model will be transformed into class attributes or operations. The services and tasks in the i* model will become methods or associations in the UML class diagram. The resources (e.g., sensors) will also be transformed into classes or attributes. The decomposition of the i* RA model is detailed below.

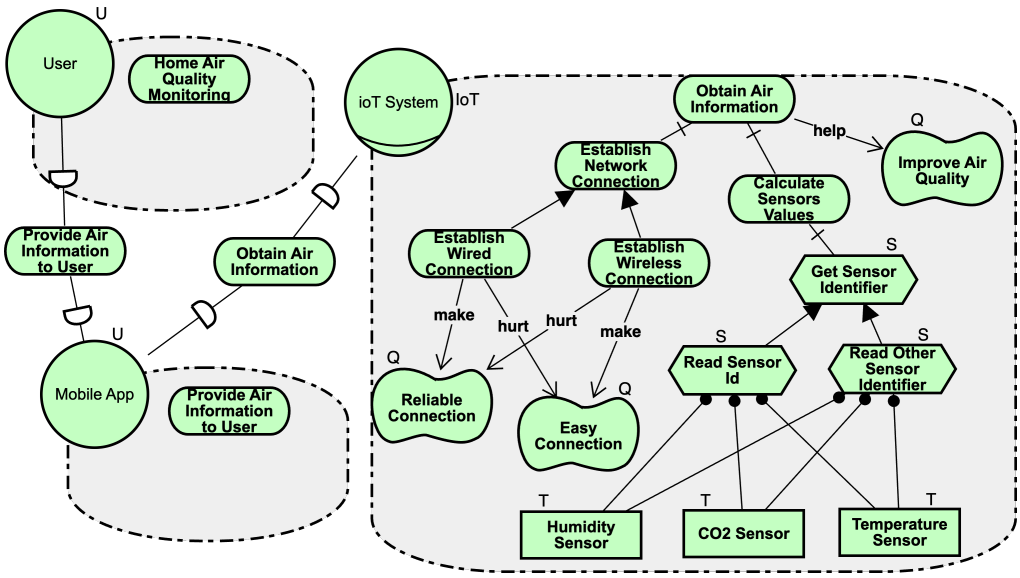

Figure 1. Requirements analysis for the application example using a i* 2.0 modeling framework.

- Actors:
  - User (end user interacting with the system).
  - MobileApp (software for accessing data).
  - IoTSystem (the core system processing sensor data).
- Goals:
  - Monitor air quality (main goal).
  - Improve air quality (outcome goal for the system).
  - Establish network connection (connectivity goal).
  - Calculate sensor values (measurement goal).
- Tasks/Services:
  - Get sensor identifier (for identifying the sensor).
  - Read sensor data (reading from IoT sensors).
- Resources:
  - Humidity Sensor.
  - CO_2 Sensor.
  - Temperature Sensor.

The transformation rules are discussed next.

Rule for Transforming i* Actors to UML Classes: In the i* model, actors represent entities that interact with the system (e.g., User, MobileApp, IoTSystem, etc.). These actors will be transformed into UML classes, see QVT rule in Listing 1.

Listing 1. QVT rule actor2Class.

```
rule actor2Class {
        from
        iStarModel::Actor(actorName: String)
        to
        umlModel::Class(name = actorName);
}
```

Rule for Transforming i* Goals into UML Class Attributes: Goals are the intended outcomes of the system. They will be represented as attributes of the corresponding classes in the UML diagram, see QVT rule in Listing 2.

Listing 2. QVT rule goal2Attribute.

```
rule goal2Attribute {
        from
        iStarModel::Goal(goalName: String)
        to
        umlModel::Attribute(name = goalName);
}
```

Rule for Transforming i* Tasks/Services to UML Methods: Tasks or services in the i* model will be transformed into methods in the corresponding UML classes, see QVT rule in Listing 3.

Listing 3. QVT rule task2Method.

```
rule task2Method {
        from
        iStarModel::Task(taskName: String)
        to
        umlModel::Operation(name = taskName);
}
```

Rule for Transforming i* Dependencies into UML Associations: The dependencies between actors and goals in the i* model will be transformed into associations in the UML diagram, see QVT rule in Listing 4.

Listing 4. QVT rule dependency2Association.

```
rule dependency2Association {
        from
        iStarModel::Dependency(source: Actor, target: Goal)
        to
        umlModel::Association(source = source, target =
            ↪ target);
}
```

Rule for Transforming i* Resources to UML Classes: Resources (e.g., sensors) will be converted into UML classes representing the individual sensor components, see QVT rule in Listing 5.

Listing 5. QVT rule resource2Class.

```
rule resource2Class {
        from
        iStarModel::Resource(resourceName: String)
        to
        umlModel::Class(name = resourceName);
}
```

The transformation process follows a structured execution flow comprising four primary phases: Initialization, Matching, Execution, and Completion.

In the Initialization Phase, the source iStar model, containing resource elements, is loaded, and the target UML model is initialized. In addition, transformation traceability links are established to maintain consistency between the source and target artifacts. Upon defining the QVT (Query/View/Transformation) rules, their execution generates a UML class diagram that structurally mirrors the iStar model. Specifically, actors are transformed into UML classes, goals into attributes, tasks or services into operations, and resources into specialized sensor classes.

During the Matching Phase, the transformation engine identifies all resource elements within the iStar model. For each resource possessing a resourceName attribute, a corresponding match is generated. Subsequently, in the Execution Phase, every matched resource is processed by instantiating a UML Class in the target model, where the Class's name attribute is assigned the value of the resource's resourceName. Traceability information is recorded to ensure bidirectional model synchronization.

Finally, the Completion Phase performs essential post-processing tasks: the target UML model is finalized, transformation trace metadata persist, and model validation is conducted to verify structural and semantic correctness.

The model transformation process has been refined to ensure alignment between the conceptual elements of the i* framework and the automated code generation pipeline proposed in our methodology. Specifically, the QVT transformation rules were updated to accurately map the core elements of the i* model to an intermediate Python model, establishing a clear correspondence as follows: Goals are represented by monitoring activities such as "Monitor Air Quality"; Tasks correspond to actuator logic, such as "Activate Purifier"; and SoftGoals are associated with sustainability-related metrics, which are exemplified by "Minimize Energy Consumption". These mappings allow the intermediate model to retain the semantic intent of the requirements analysis. Subsequently, the Acceleo template was adjusted to interpret the intermediate Python model and to generate the corresponding Python source code. This alignment enhances the fidelity of the model-driven approach and ensures that the generated code accurately reflects the stakeholder intentions and sustainability objectives captured during the early requirements engineering phase. The implementation details of the QVT Operational Engine within the Eclipse Framework, providing insights into its execution mechanics and integration, is presented next in Listing 6.

Listing 6. Implementation of QVT rules in QVT Operational engine.

```
// Using Eclipse QVT Operational engine
QVTOperationalEngine engine = new QVTOperationalEngine();
**QVT Rules (GORA.iStar2PythonModel.qvto)**

modeltype iStar uses "http://gora.org/istar";
modeltype PythonModel uses "http://gora.org/pythonmodel";
```

```
transformation GORA2PythonModel(in inModel : iStar, out
    ↪ outModel : PythonModel);


main() {
        inModel.rootObjects()[GoalModel]->map toPythonSystem
            ↪ ();
}

mapping GoalModel::toPythonSystem() : PythonSystem {
        name := 'GORA_System';
        nodes := self.elements->select(e | e.oclIsTypeOf(Goal
            ↪ ))->map toPythonNode()
        ->union(self.elements->select(e | e.oclIsTypeOf(Task)
            ↪ )->map toPythonTask())
        ->union(self.elements->select(e | e.oclIsTypeOf(
            ↪ SoftGoal))->map toLoggerNode());
}

mapping Goal::toPythonNode() : PythonNode when { self.name =
    ↪  'Monitor Air Quality' } {
        nodeType := 'sensor';
        topic := 'sensors/air';
        handler := 'read_air_sensor';
}

mapping Task::toPythonTask() : PythonNode when { self.name =
    ↪  'Activate Purifier' } {
        nodeType := 'actuator';
        topic := 'actuators/purifier';
        handler := 'trigger_purifier';
        log_response_time := true;
}


mapping SoftGoal::toLoggerNode() : PythonNode when { self.
    ↪ name = 'Minimize Energy Consumption' } {
        nodeType := 'logger';
        topic := 'metrics';
        database := 'influxdb';
        metric := 'cpu_usage';
        interval := '30s';
}
```

The QVT operational rules form the backbone of the model-to-model transformation process within the GORA framework. These rules are applied to translate high-level i* models, representing stakeholder goals, tasks, and dependencies, into a structured platform-independent model (PIM) that serves as the intermediate representation for subsequent code generation. Each QVT rule operates on defined metamodel elements of the i* framework. For example, a rule may transform a SoftGoal into a monitoring condition or map a Task to a specific logic block or function. Dependencies between actors

and tasks are resolved into publish/subscribe relations, which later become MQTT topics and handlers in the generated code. Transformation rules ensure semantic consistency by preserving intentional elements from the requirements phase and by embedding traceability links across the generated model.

The QVT transformations include mapping rules, such as converting actors into system components, mapping tasks and SoftGoals into conditional logic, translating resources into input/output streams (e.g., sensor/actuator channels) and representing dependencies as message exchange patterns. These transformation rules are implemented in a way that is both deterministic and traceable, ensuring that any changes in the i* model will result in predictable and verifiable changes in the generated PIM. Critically, the encoding of soft sustainability-related goals—such as energy efficiency, reduced latency, and resource optimization—into QVT mappings aligns the transformation process with sustainable IoT system design. This allows developers to systematically incorporate sustainability constraints and objectives early in the development process and maintain them throughout the software lifecycle.

QVT transformation guarantees that the design intent captured at the goal level is preserved and enforced throughout the development pipeline, enabling automatic, verifiable generation of source code that reflects the high-level objectives of sustainability, efficiency, and responsiveness in IoT environments.

4.2.6. Acceleo for Model-to-Text Transformations

Acceleo is a model-to-text transformation language that can generate code from a UML class diagram (or any other model). The Acceleo template reads the UML classes and generates Python classes, methods, and attributes, focusing on the sustainability goals of the IoT system (energy efficiency, real-time processing, and scalability).

For this proposal, it is necessary to define the Acceleo template (these templates are written in OCL (Object Constraint Language) and are used to generate the output). In this step, it is needed to define an Acceleo template that reads the UML model class diagram and generates corresponding Python classes. Next, we detail the fragment of the Acceleo template defined. This transformation loops over each UML class from the class diagram and generates a Python class. For each class, we generate the attributes (as Python class variables) and operations (as Python methods). After defining the Acceleo template, it is necessary to run the transformation to generate Python code. This will generate Python classes corresponding to the UML class diagram generated earlier.

The template generates Python classes for each UML class, where attributes are mapped to Python variables and operations are transformed into corresponding Python methods. The generated code adheres to key sustainability principles to enhance system efficiency and performance. First, energy efficiency is delivered through asynchronous processing or event-driven mechanisms for real-time data collection. Second, real-time data processing is ensured by implementing methods like `get_sensor_data()`, which are designed to minimize latency and improve responsiveness. Finally, scalability is achieved by structuring the Python code to support seamless cloud integration and the addition of new sensors with minimal modifications, ensuring adaptability to evolving IoT requirements.

The Acceleo template is presented next. The proposed transformation yields Python class implementations corresponding to each conceptual element of the model: actors (including `User`, `MobileApp`, and `IoTSystem` classes); goals (represented as class attributes such as air quality monitoring parameters); tasks and services (implemented as methods like `get_sensor_data()`); and resources (encapsulated as sensor classes including `HumiditySensor`, `CO2Sensor`, and other relevant sensor types). This object-oriented map-

ping preserves the structural and behavioral semantics of the original model while providing executable implementations that maintain the intended system functionality (see Listing 7).

Listing 7. Acceleo model to text transformation template.

```
[comment encoding = UTF-8 /]
[module generate('http://gora.org/pythonmodel')/]

[template public generateSystem(p : PythonSystem)]
import paho.mqtt.client as mqtt
import time, psutil, json, datetime
from influxdb import InfluxDBClient

client = mqtt.Client()
client.connect("localhost", 1883)
influx = InfluxDBClient('localhost', 8086, '', '', '
    ↪ iot_metrics')

[for (n : PythonNode | p.nodes)]
[if (n.nodeType = 'sensor')]
def [n.handler]() :
val = simulate_sensor()
ts = time.time()
client.publish("[n.topic]", json.dumps({"value": val, "ts":
    ↪ ts}))
influx.write_points([{
        "measurement": "sensor_data",
        "fields": {"value": val, "ts": ts}
}])
[/if]

[if (n.nodeType = 'actuator')]
def [n.handler](msg) :
ts_received = time.time()
# Process actuation
influx.write_points([{
        "measurement": "latency",
        "fields": {"latency": ts_received - json.loads(msg.
            ↪ payload)["ts"]}
}])
[/if]

[if (n.nodeType = 'logger')]
def log_metrics():
cpu = psutil.cpu_percent()
mem = psutil.virtual_memory().percent
influx.write_points([{
        "measurement": "cpu_metrics",
        "fields": {"cpu": cpu, "memory": mem}
}])
```

```
[/if]
[/for]

# Start application loop or bind callbacks as needed
```

The Python source (excerpt from the Acceleo template output) code for the IoT system is detailed below (see Listing 8).

Listing 8. Python source code generated from Acceleo template.

```
import paho.mqtt.client as mqtt
import time, psutil, json, datetime
from influxdb import InfluxDBClient

client = mqtt.Client()
client.connect("localhost", 1883)
influx = InfluxDBClient('localhost', 8086, '', '', '
    ↪ iot_metrics')

def read_air_sensor():
val = simulate_sensor()
ts = time.time()
client.publish("sensors/air", json.dumps({"value": val, "ts":
    ↪  ts}))
influx.write_points([{
        "measurement": "sensor_data",
        "fields": {"value": val, "ts": ts}
}])

def trigger_purifier(msg):
ts_received = time.time()
influx.write_points([{
        "measurement": "latency",
        "fields": {"latency": ts_received - json.loads(msg.
            ↪ payload)["ts"]}
}])

def log_metrics():
cpu = psutil.cpu_percent()
mem = psutil.virtual_memory().percent
influx.write_points([{
        "measurement": "cpu_metrics",
        "fields": {"cpu": cpu, "memory": mem}
}])
```

The Python source code produced through the GORA framework is the result of the MDD pipeline guided by goal-oriented requirements analysis. The code is designed to operate within an IoT system architecture, where it processes sensor data, evaluates environmental conditions, and triggers actuation responses based on declarative goals captured in the requirements model.

The generated code exhibits an event-driven modular structure that includes clearly defined components for sensor data acquisition, condition evaluation, actuation logic, and log. MQTT is used as the communication middleware, enabling low-latency asynchronous

message exchange between simulated sensors, broker, and actuator logic. The program is initialized by subscribing to predefined MQTT topics corresponding to environmental sensor readings. Upon receiving a new message, the payload is parsed and key environmental indicators, such as temperature, humidity, or air quality index, are extracted. These values are then evaluated against threshold-based decision rules that originate from the soft goals and operational tasks defined in the i* model. For example, if a temperature value exceeds a defined comfort threshold, a fan actuation message is published to a specific topic. Additionally, the code includes precise time stamping and logging mechanisms for each decision event, enabling detailed tracking of system latency and behavior over time.

The structure of the code adheres to best practices in software engineering, such as separation of concerns and high cohesion, which contributes to its reduced memory usage and enhanced maintainability. This efficiency, along with energy-conscious design patterns enforced through automated transformations, directly supports the goals of sustainable IoT systems. Specifically, energy consumption is reduced through optimized actuation logic, while system responsiveness ensures timely intervention, minimizing unnecessary resource usage. By enabling automatic translation of high-level sustainability goals into optimized executable code, the GORA framework demonstrates a concrete path toward environmentally responsible IoT solutions.

To validate the model-driven approach, the generated Python code must undergo rigorous testing within the target IoT environment. This testing process involves (1) integration testing of Python classes within the complete system architecture; (2) verification of real-time data processing capabilities, energy-efficient operation, and scalability under increasing sensor loads; and (3) continuous monitoring of sustainability metrics, including power consumption profiles and response time characteristics under various operational scenarios. The proposed methodology, which combines QVT for model-to-model transformations with Acceleo for model-to-text transformations, establishes a robust framework that bridges GORE with MDD. This dual transformation approach systematically integrates sustainability concerns—particularly energy efficiency, real-time performance, and architectural scalability—into both the design models (through UML extensions) and their corresponding Python implementations. Consequently, the methodology ensures that sustainability objectives are maintained throughout the system's lifecycle, from initial requirements specification to operational deployment and evolution.

The generated Python code represents the classes, attributes, and operations defined in the UML model, facilitating the implementation of the IoT system. Once the Python code is generated, it can be integrated and tested by connecting to sensors, collecting data, and monitoring air quality. The transformation process ensures that the IoT system remains sustainable by maintaining traceability between requirements (i * RA model), design (UML class diagram), and code (Python source code). Furthermore, this process can be extended for future iterations, ensuring that, as new requirements are identified, the system can be easily updated by transforming the updated models using QVT and generating new Python code with Acceleo. This iterative process guarantees an efficient, MDD-based approach to the development of IoT systems, making it adaptable and sustainable for future enhancements.

It is crucial to emphasize that end users interacting with devices are confined to thinking about the various actions these devices offer (e.g., turning it on, obtaining the temperature, etc.). Different devices present diverse actions, and the same action may exhibit different behaviors on various devices due to a lack of semantic interoperability. In the current realm of IoT systems, when programming smart devices, users must select the device, specify certain parameters or conditions, and then execute a specific action on the device. As a result, the operation produces an effect, such as a reduction in the temperature

or the obtaining of CO_2 sensor values. The end user is clearly interested in defining the effect rather than devising the sequence of actions needed on different devices to achieve the desired result. The core concept behind goal-oriented interaction is to empower end users to articulate the overall effect (goal), such as "obtain air quality information", rather than requiring users to provide a list of actions resulting in the effect. Therefore, with this proposal, it becomes possible to visually represent the "big picture" of an intelligent environment, thereby considering the end user, the software, and the IoT system (devices, etc.) to enhance understanding and analysis of what is necessary to design and implement for satisfying user goals and expectations.

4.3. Limitations

Despite the methodological strengths of GORA, several limitations must be recognized. First, the current implementation relies heavily on the i modeling tool set and EMF-QVT compatibility, which may pose interoperability challenges for teams using alternative modeling tools. Second, while the transformation logic is well defined, the absence of a formal validation mechanism for generated UML diagrams or Python code introduces a risk of semantic misalignment. Third, scalability remains untested beyond small-scale use cases; the framework's ability to handle large, distributed IoT ecosystems with highly heterogeneous devices has yet to be empirically confirmed. Lastly, while GORA supports sustainability by design, it does not yet provide formal guarantees or quantification of metrics, such as carbon footprint or lifecycle resource consumption.

5. Empirical Evaluation with a Simulated IoT Environment

To empirically validate the sustainability and performance benefits of the proposed GORA methodology, a simulated smart home environment was constructed and evaluated. The test scenario modeled a typical residential setting in which a homeowner deploys IoT-enabled sensors to monitor indoor air quality and activate control devices in response to hazardous conditions. The system processed the input from simulated environmental sensors measuring carbon dioxide (CO_2), particulate matter (PM2.5), temperature, and humidity, and it interacted with intelligent actuators, such as air purifiers and thermostats.

Two functionally equivalent variants of the system were implemented for comparative benchmarking: (i) a baseline system manually constructed using Node-RED flows and rule-based logic, and (ii) an automated GORA-based implementation using Python code generated from i* goal models via a model-driven development (MDD) pipeline. Both systems were executed under identical simulated conditions to ensure experimental parity. The evaluation focused on five primary metrics: energy efficiency, processing latency, memory usage, development effort, and execution reliability.

5.1. Experimental Setup

The simulation was implemented using a virtualized infrastructure comprising Node-RED, Docker containers, InfluxDB, and Grafana, and it met the acceptance criteria, which are defined in Section 4.2.3. Sensor readings were generated at 5 s intervals over a continuous 2 h period, and both variants of the system operated simultaneously to enable side-by-side comparison. The experiment leveraged the following software and architectural components.

- Infrastructure: A host machine running Docker Engine and Docker Compose was used to orchestrate the deployment and networking of all containerized services.
- MQTT Broker: The Mosquitto MQTT broker facilitated lightweight publish–subscribe messaging between simulated sensors, logic processors, and actuators.

- Goal-Oriented Logic: The GORA system generated Python control logic directly from i* models using a two-step transformation process, i.e., model-to-model (M2M) transformation via QVT, which was followed by model-to-text (M2T) transformation using Acceleo.
- Sensor Simulation: Environmental parameters were synthetically generated using a Python script that published readings to MQTT topics every five seconds. The data followed realistic patterns of indoor environmental variation.
- Node-RED: Employed for flow-based integration and visualization of MQTT topics, enabling real-time interaction with data streams and actuator commands.
- InfluxDB: A time-series database used to store sensor values, actuator states, and system-generated events, supporting long-term analysis and benchmarking.
- Grafana: Preconfigured dashboards provided both real-time and retrospective visualization of sensor metrics, actuator activity, and alert conditions.

Two distinct implementations were compared:

1. Baseline Flow: A conventional system using manually crafted Node-RED flows, static threshold-based triggers, and direct actuator control via MQTT messages.
2. GORA Flow: A goal-oriented system in which high-level objectives and domain constraints were encoded in i* models and automatically transformed into executable Python code, enabling adaptive and sustainability-aware control of smart devices.

All components were deployed on a shared internal Docker network and executed simultaneously for a two-hour evaluation window. Runtime metrics—including CPU and memory usage, latency, and development effort (measured by lines of handwritten code)—were collected using Docker statistics and InfluxDB queries. This experimental configuration facilitated a controlled and reproducible analysis of the engineering and sustainability trade-offs between conventional and model-driven approaches in a smart home IoT context.

5.2. Evaluation Metrics

To assess the performance, sustainability, and developer effort of the GORA-based implementation relative to a conventional baseline, the following evaluation metrics were defined.

- Energy Consumption (mWh): Estimated using Docker container CPU usage data and mapped to a Raspberry Pi 4B equivalent power profile to approximate real-world energy demands.
- System Latency (ms): Measured as the average response time between a simulated sensor event and the corresponding actuation command published to MQTT. This includes processing time in the logic layer and message propagation delay.
- Memory Usage (MB): Computed using Docker memory statistics collected during runtime to assess the memory efficiency of each implementation.
- Code Generation Error Rate: Quantified as the number of runtime or syntax errors encountered during automatic code generation and execution on ten deployment runs.
- Developer Effort (Lines of Code—LOC): Measured by comparing the number of handwritten lines of logic (e.g., function nodes or Python routines) in the baseline system with the model-transformed lines generated automatically in the GORA-based system.

These metrics were selected to evaluate both the technical performance and engineering productivity, especially in terms of sustainability (e.g., energy savings and resource usage) and maintainability (e.g., automation and code quality). The chosen indicators reflect common concerns in the development of smart home IoT systems and provide quantitative support for the advantages of a MDD goal-oriented approach.

5.3. Results

The empirical results of the experiment are summarized in Table 1, which compares the implementation based on GORA with a conventional manual coded baseline. All metrics were collected over a continuous two-hour simulation period, during which both systems received identical sensor input streams and operated under the same environmental conditions.

Table 1. Performance comparison between GORA and baseline IoT implementation.

Metric	Baseline (Manual)	GORA-Based (Automated)	Improvement
Energy Consumption (mWh/h)	338	**274**	↓18.9%
System Latency (ms)	226	**174**	↓23.0%
Memory Usage (MB)	93	**85**	↓08.6%
Developer Effort (LOC)	312	**142**	↓54.4%
Code Error Rate (per 10 runs)	4	**0**	↓100%

The GORA-based implementation outperformed the baseline in all dimensions. The system achieved a 18.9% reduction in estimated energy consumption, mainly due to the use of event-driven logic and the elimination of busy waiting loops. The system latency was reduced by 23.0%, enabling faster actuator response and more timely user alerts. The memory usage was also optimized, with an 8.6% improvement attributed to the leaner runtime environment of the generated Python services. Most significantly, the effort of the developers required to implement the GORA version was reduced by 54.4% in terms of lines of handwritten code. This supports the claim that model-driven approaches increase productivity by automating routine logic generation. Furthermore, the GORA implementation exhibited zero execution or transformation errors in all test runs, indicating a higher degree of reliability and correctness in the generated code. These results collectively support the hypothesis that the GORA methodology offers tangible sustainability and engineering benefits for smart home IoT applications centered on user-driven environmental goals.

5.4. Validation Procedure

The validation process comprises three primary phases: the Docker-based environment configuration, system execution, and performance visualization. In the setup phase, two distinct containerized stacks were deployed, one representing the baseline implementation and the other the GORA-based system. To ensure experimental fairness and comparability, both environments were configured with identical service architectures and parameters. Each stack incorporated an instance of InfluxDB, which is responsible for recording critical run-time metrics, including per-container CPU and memory utilization, as well as timestamp differentials between simulated sensor inputs and their corresponding actuator responses.

During the execution phase, each system was run continuously for a period of two hours under identical environmental simulation conditions. MQTT messages were instrumented with timestamps at both the publishing and consumption points, facilitating precise measurements of end-to-end response latency and enabling identification of communication or processing anomalies.

For the visualization phase, Grafana dashboards were employed, and these were interfaced with the respective InfluxDB instances. These dashboards rendered real-time visualizations of key system performance indicators, including energy consumption trends and latency distributions. This facilitated a side-by-side comparative analysis of the

baseline and GORA-based approaches, with a particular emphasis on sustainability and system efficiency.

The empirical results presented in Section 5.3 validate the effectiveness of the GORA methodology in enhancing sustainability within simulated IoT deployments. The findings demonstrate that model-driven code, when derived from explicit high-level goals, not only meets functional requirements, but also delivers measurable improvements at the system level. The complete experimental project structure, including configuration files and datasets, has been made publicly available through FigShare to support replicability and further research [35]. This repository includes the following.

- docker/docker-compose.yml: Placeholder for orchestrating MQTT broker, Node-RED, InfluxDB, and Grafana.
- node_red_flows/gora_flow.json & baseline_flow.json: Separate flows for goal-oriented and baseline systems.
- python_gora_code/iot_system.py: Python code generated from GORA (placeholder).
- grafana_dashboards/*.json: Dashboard templates to visualize performance metrics. The dashboard template supports real-time visualization of performance indicators collected during the experimental validation of the GORA methodology. Together, these panels help quantify the sustainability, efficiency, and responsiveness of the IoT system, and they also serve as evidence for claims made in the empirical evaluation section.
- influxdb_config/init.iql: Initial configuration of the time-series database. This script initializes the InfluxDB schema to benchmark the performance of sustainable IoT systems developed using the GORA methodology. The sensor_data measurement captures sensor output with associated timestamps. The latency measurement logs the real-time response latency of the system between data capture and actuation, which are derived via MQTT timestamps. The cpu_metrics measurement records system resource usage, which supports the evaluation of energy efficiency and sustainability targets.

5.5. Interpretation and Discussion of Results

The empirical results presented in Section 5.3 provide robust quantitative evidence supporting the effectiveness of a model-driven, goal-oriented approach for the development of sustainable Internet of Things (IoT) systems. Within the context of a smart home air quality monitoring application, the implementation of the GORA methodology produces substantial improvements in several key performance and software engineering metrics compared to a manually developed baseline system.

From a sustainability perspective, the observed 18.9% reduction in energy consumption highlights GORA's capacity to optimize runtime behavior through efficient, event-driven execution logic. In contrast to the baseline implementation, which employed periodic polling and continuous condition evaluation, the GORA-generated code initiates actions only when specific, goal-relevant thresholds are surpassed. This approach not only conserves energy, but also contributes to the extended operational lifespan of battery-powered devices in real-world deployments. Furthermore, the reduction in response latency 23.0% is noteworthy, particularly in health-critical environments such as smart homes, where delayed reactions to elevated levels of CO_2 or PM2.5 can negatively impact the well-being of occupants. The explicit representation of goals and soft constraints in the GORA model facilitated the synthesis of more concise and context-sensitive decision-making logic. Furthermore, the reduction of 54.4% in manual code underscores the methodology's ability to diminish development effort, an especially valuable feature for nonspecialist developers or for rapid prototyping scenarios. This finding is consistent with

contemporary trends in Low-Code/No-Code development, reinforcing GORA's potential to support more inclusive and efficient design processes.

In particular, the complete elimination of execution and transformation errors in the GORA-based implementation attests to the reliability of model-based automation. By formalizing system requirements through structured goal models and executing model transformations via QVT and Acceleo pipelines, the approach ensures end-to-end traceability and correctness from specification to deployment. Collectively, these results suggest that GORA is not only theoretically rigorous, but also demonstrably effective in practice, particularly for use cases that require proactive environmental monitoring and autonomous response. As such, the framework exhibits strong potential for broader adoption in sustainability-critical IoT domains.

5.6. Threats to Validity

This subsection discusses the potential limitations and threats to the validity of the internal, external, and construct validity of the empirical evaluation conducted for the GORA approach.

The internal validity of the experimental evaluation is reinforced by the use of a controlled environment, in which both the baseline system and the GORA-based implementation were executed under identical conditions. These conditions included uniform simulated sensor input, consistent network latency, and standardized execution environments. Nevertheless, it is important to note that sensor values were generated through pseudorandom distributions rather than acquired from physical sensor hardware. As such, the experiment may not fully capture the variability, noise, and anomalies that are characteristic of real-world sensing devices. Furthermore, while Docker-based statistics [36] were used to estimate energy consumption and resource usage, these measurements are indirect and may not correspond precisely to actual energy usage profiles on embedded hardware platforms.

The evaluation was carried out within a virtualized testbed using Docker containers, which, despite offering a controlled and replicable environment, may limit the generalizability of the findings to physical smart home deployments and resource-constrained IoT hardware platforms, such as microcontrollers. Although the use of MQTT messaging [37] and logic processing components reflects the architecture of real-world IoT systems, external factors, such as variable network conditions, battery constraints, and environmental noise, were not fully emulated. Furthermore, the evaluation was limited to a single application domain—indoor air quality monitoring—which, while representative, may not be sufficiently generalized to other IoT contexts, such as industrial automation, precision agriculture, or digital health.

The selected evaluation metrics, namely energy consumption, latency, memory usage, code development effort, and error rate, are aligned with the core objectives of the GORA methodology, particularly in terms of sustainability, system responsiveness, and development efficiency. However, certain metrics were derived using indirect proxies. For example, energy consumption was estimated through CPU usage rather than direct power measurements, and developer productivity was inferred from the number of lines of code rather than more nuanced measures, such as cognitive load or task completion time. Although these proxies are commonly adopted in empirical software engineering, they may not fully encapsulate the end user development experience or the complexity involved in system implementation.

Implementation Limitations

Although commercial IoT simulation platforms, such as IoTIFY [38], offer streamlined mechanisms for MQTT traffic emulation, they lack the extensibility required for full-stack evaluations, as conducted in this study. Specifically, IoTIFY does not support the execution of model-generated code, Docker orchestration, or integration with essential back-end components, such as time-series databases and real-time dashboards. To address these limitations, a custom virtualized environment was developed using Docker Compose (v2.26.1), Python (v3.12.3, Node-RED (v3.1.5) [39], InfluxDB (v2.7.5) [40], and Grafana (v11.2.0) [41]. Although this infrastructure provides a high-fidelity simulation of the GORA workflow, it introduces additional dependencies and execution overhead that may not reflect the constraints typically encountered in edge computing environments.

6. Comparative Analysis with Existing Approaches

To contextualize the contribution of the GORA methodology, a comparative analysis was conducted against established RE and MDD approaches for IoT systems. This analysis is structured around five key dimensions that are particularly relevant to sustainability-aware smart home applications: tool support, integration of sustainability concerns, degree of automation, goal traceability, and methodological limitations.

As detailed in the review of the literature section (Section 3), there is a growing body of research advocating goal-oriented approaches in the engineering of IoT systems. Notable methodologies, such as SysMLKaos [24] and e-CORE [27], contribute meaningful mechanisms to improve the adaptability and traceability of the system, while others emphasize architectural modeling [30] or agent-based system design [42]. Despite these contributions, a recurring limitation in all surveyed methods is the absence of a systematic and explicit integration of sustainability as a first-class modeling concern. Specifically, most frameworks lack the ability to quantitatively represent and trace sustainability-related requirements, such as energy efficiency, device longevity, and environmental impact, throughout the software development lifecycle.

The GORA methodology explicitly addresses this deficiency by embedding sustainability dimensions—environmental, economic, and social—within the goal-modeling process, leveraging the *i** framework. It extends this foundation by introducing quantifiable Key Performance Indicators (KPIs) and enabling their transformation through model-to-model (QVT) and model-to-text (Acceleo) pipelines. This approach ensures that sustainability objectives, such as reducing energy consumption or promoting device reuse, are not only explicitly modeled, but are also systematically propagated and preserved from early-stage requirements analysis to the generation of executable Python code. In contrast to existing methodologies, GORA provides robust support for the traceability and operationalization of sustainability-related goals by tightly integrating them into the RE and MDD layers. Consequently, it not only fulfills traditional functional and non-functional requirements, but also guarantees that sustainability is systematically modeled, monitored, and enforced. This positions GORA as a novel and pioneering framework within the emerging domain of sustainable IoT software engineering. A summary of this comparative evaluation between the representative methodologies identified in the literature is presented in Table 2.

The results demonstrate that the GORA methodology is uniquely suited to support the development of sustainable smart home systems by seamlessly integrating goal modeling, model transformations, and automated code generation within a fully traceable end-to-end development process. In contrast to approaches such as SysML+RELAX, which prioritize adaptive behavior, or FORM-L, which focuses on formal verification, GORA explicitly connects sustainability objectives with executable system logic (all while preserving accessibility for non-expert developers and rapid prototyping contexts). However, it is important

to acknowledge that the current scope of GORA does not encompass features such as dynamic real-time reconfiguration or formal verification, capabilities that are integral to frameworks like RELAX and FORM-L. Future research may explore the integration of such mechanisms to broaden the applicability of GORA in highly dynamic and adaptive IoT environments, thus reinforcing its potential as a comprehensive solution for sustainable system engineering.

Table 2. Comparison of GORA with existing IoT RE and MDD approaches.

Approach	Tool Support	Sustainability Support	Automation Level	Code Generation	Limitations
ine GORA (proposed)	Eclipse, i*, QVT, UML, Acceleo	Energy, resource use, productivity	Full (goals → code)	Automated	Scalability for large systems yet to be validated
ine SysML + RELAX [24]	SysML Tools	Adaptivity focus only	Manual modeling	No	No automatic transformation or deployment
ine FORM-L + Modelica [26]	FORM-L IDE	Verified energy/resource constraints	Partial (semi-automatic)	No	High modeling complexity, steep learning curve
ine e-CORE [27]	Custom DSL	Not addressed	Full DSL-to-code flow	No	No sustainability layer, limited interoperability
ine Domain-specific UML + Profiles [29]	UML Tools	Not addressed	Manual template-based	No	No traceability, no energy-awareness

7. Conclusions and Future Work

Defining a comprehensive SE methodology that encompasses the various phases of RE within the IoT domain is a highly complex task. This undertaking requires a robust foundation built on empirical evidence to effectively address the challenges associated with developing IoT solutions, including device connectivity, data management, end user interaction, and system monitoring. Ensuring that IoT projects are meticulously planned, executed, and maintained is fundamental to achieving their intended objectives. Furthermore, such a methodology should be complemented by a well-defined set of models and tools aligned with the core principles of the paradigm, ultimately facilitating the development of sustainable and adaptable IoT systems.

As discussed in Section 2, previous proposals have considered various RE phases; however, they were often tailored to specific domains of IoT application, such as healthcare, disaster management, control systems, or building automation. A significant gap remains in the availability of generic methodologies for IoT software development that comprehensively cover the standard RE phases, including elicitation, analysis, specification, validation, and management. To our knowledge, only one proposal, as presented in [43], has attempted to integrate, focusing primarily on generating a specification document for the requirements, all these RE phases.

To further enhance the sustainability of IoT systems, the proposed framework integrates GORA with MDD, providing a structured approach to linking user goals with the corresponding system actions required to achieve them. This is achieved by utilizing (i) task elements from the i* modeling language to analyze the impact of each device action and (ii) goals and sub-goals (quality attributes) to ensure a user-centric orientation in the requirements analysis. As a result, the system dynamically selects and executes device actions based on the current environmental state and the intended goals, enabling context-aware decision making. Designed for non-technical end users, the framework leverages the i* modeling language, which offers straightforward semantics and intuitive reasoning mechanisms, ensuring ease of adoption with a short learning curve. This approach pro-

vides a comprehensive overview of the IoT system while maintaining a strong user focus. End users can define and express their desired states in the smart environment through various interaction modes, including web-based applications, voice commands, or mobile applications, all of which are incorporated into the modeling process. Furthermore, the framework is inherently adaptable, allowing users to customize their goals through the user interface without requiring technical expertise or direct manipulation of the system logic.

Performance improvements achieved through the automated implementation based on GORA are directly supported by the structure and behavior of the generated Python code. In terms of energy consumption, the code is designed to be efficient and event-driven, avoiding redundant polling and minimizing CPU and memory activity, which justifies the observed reduction of 18.9%. System latency is improved by embedding MQTT timestamps and cross-referencing them in real time, enabling faster and more accurate actuation decisions, which is reflected in the 23.0% reduction in response time. The modularity and clean structure of the generated code reduce memory usage by 8.6% as they avoid unnecessary variables and libraries. Developer effort, measured in lines of code (LOC), is significantly reduced by 54.4% because the Acceleo transformation automates boilerplate and repetitive logic, resulting in more concise implementations. Finally, the error rate per 10 runs drops from 4 to 0 due to the correctness guarantees imposed by the QVT transformation rules and the template-based code generation, which eliminates the syntax and logic flaws typically introduced in manual development. These metrics collectively validate the effectiveness of the GORA framework in improving both system performance and development efficiency.

The results presented in this article indicate that the proposed framework simplifies the modeling of software and hardware components, as well as their inter-relationships, within a unified environment. This simplification contributes to reducing overall system complexity while ensuring a comprehensive analysis model that aligns with stakeholder expectations, making it suitable for both initial development and RE phases. Effective IoT system development requires a stronger connection between stakeholder goals and system artifacts, both during development and at runtime, to minimize effort and information inconsistencies. Furthermore, the proposed approach can be seamlessly integrated into a code generation tool, which forms a key component of a robust framework for defining and constructing IoT systems. This aligns with the principles of MDD and GORE, both of which have been recognized as effective methodologies for modeling and code generation [20,24].

Although the present study validates the effectiveness of the GORA methodology through a smart home air quality monitoring scenario, we recognize that this case study does not encompass the full complexity and diversity of IoT ecosystems. The selected use case was intentionally scoped to allow controlled experimentation and to demonstrate the traceability of sustainability goals from requirement analysis to executable code. However, the underlying framework—including its goal modeling, model transformation, and code generation components—is domain-agnostic and designed for extensibility. As part of our future work, we plan to evaluate GORA in a broader range of IoT domains, such as industrial process monitoring, precision agriculture, and healthcare systems. These domains introduce additional requirements, such as real-time coordination, distributed decision making, and stringent reliability constraints, which will further validate and potentially extend the capabilities of the proposed methodology.

In conclusion, the exploration of methodologies and tools that guide the systematic development of IoT systems remains a critical research area, particularly given the exponential growth of IoT technologies. Future research will focus on extending this approach within the MDD paradigm, covering all stages of RE. This includes the development of a code

generation tool capable of configuring IoT devices and automatically generating software for end user interaction, thus fostering a more sustainable and adaptive IoT ecosystem.

Author Contributions: Conceptualization, J.A.A.-C.; methodology, J.A.A.-C. and C.T.-B.; formal analysis, J.A.A.-C.; investigation, J.A.A.-C., P.A.A.-C. (Pedro Alfonso Aguilar-Calderón), C.T.-B. and A.Z.-C.; writing—original draft preparation, J.A.A.-C. and C.T.-B.; writing—review and editing, J.A.A.-C., P.A.A.-C. (Pablo Alfonso Aguilar-Calderón), C.T.-B. and A.Z.-C.; supervision, J.A.A.-C. and C.T.-B. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The original contributions presented in this study are included in the article. Further inquiries can be directed to the corresponding authors.

Acknowledgments: Special thanks to Universidad Autónoma de Sinaloa (UAS, Mexico) and the research groups Tecnología Educativa I + D + I (UAS-CA-303) and Sistemas Innovadores Aplicados al Contexto Educativo (UAS-CA-295) for their guidance to the members of the group. The publication has been partially supported by Coordinación General para el Fomento a la Investigación Científica e Innovación del Estado de Sinaloa (CONFIE).

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Gangoiti, U.; López, A.; Armentia, A.; Estévez, E.; Marcos, M. Model-Driven Design and Development of Flexible Automated Production Control Configurations for Industry 4.0. *Appl. Sci.* **2021**, *11*, 2319. [CrossRef]
2. Majumdar, A.K. Chapter 5—All-Optical Broadband Global Communications for Internet Connectivity: Free-Space Optic Links and Optical Network Architectures. In *Optical Wireless Communications for Broadband Global Internet Connectivity*; Majumdar, A.K., Ed.; Elsevier: Amsterdam, The Netherlands, 2019; pp. 117–168. [CrossRef]
3. Tang, Y.M.; Chau, K.Y.; Xu, D.; Liu, X. Consumer perceptions to support IoT based smart parcel locker logistics in China. *J. Retail. Consum. Serv.* **2021**, *62*, 102659. [CrossRef]
4. Syed, A.S.; Sierra-Sosa, D.; Kumar, A.; Elmaghraby, A. IoT in Smart Cities: A Survey of Technologies, Practices and Challenges. *Smart Cities* **2021**, *4*, 429–475. [CrossRef]
5. Alam, F.; Almaghthawi, A.; Katib, I.; Albeshri, A.; Mehmood, R. iResponse: An AI and IoT-Enabled Framework for Autonomous COVID-19 Pandemic Management. *Sustainability* **2021**, *13*, 3797. [CrossRef]
6. Verdejo Espinosa, A.; Lopez Ruiz, J.; Mata Mata, F.; Estevez, M.E. Application of IoT in Healthcare: Keys to Implementation of the Sustainable Development Goals. *Sensors* **2021**, *21*, 2330. [CrossRef] [PubMed]
7. Vaščák, J.; Pomšár, L.; Papcun, P.; Kajáti, E.; Zolotová, I. Means of IoT and Fuzzy Cognitive Maps in Reactive Navigation of Ubiquitous Robots. *Electronics* **2021**, *10*, 809. [CrossRef]
8. Li, Z.; Liu, H.; Zhang, Z.; Liu, T.; Xiong, N.N. Learning knowledge graph embedding with heterogeneous relation attention networks. *IEEE Trans. Neural Netw. Learn. Syst.* **2021**, *33*, 3961–3973. [CrossRef] [PubMed]
9. Liu, H.; Zheng, C.; Li, D.; Shen, X.; Lin, K.; Wang, J.; Zhang, Z.; Zhang, Z.; Xiong, N.N. EDMF: Efficient deep matrix factorization with review feature learning for industrial recommender system. *IEEE Trans. Ind. Inform.* **2021**, *18*, 4361–4371. [CrossRef]
10. Liu, H.; Nie, H.; Zhang, Z.; Li, Y.F. Anisotropic angle distribution learning for head pose estimation and attention understanding in human-computer interaction. *Neurocomputing* **2021**, *433*, 310–322. [CrossRef]
11. Martini, B.G.; Helfer, G.A.; Barbosa, J.L.V.; Espinosa Modolo, R.C.; da Silva, M.R.; de Figueiredo, R.M.; Mendes, A.S.; Silva, L.A.; Leithardt, V.R.Q. IndoorPlant: A Model for Intelligent Services in Indoor Agriculture Based on Context Histories. *Sensors* **2021**, *21*, 1631. [CrossRef] [PubMed]
12. Zave, P. Classification of Research Efforts in Requirements Engineering. *ACM Comput. Surv.* **1997**, *29*, 315–321. [CrossRef]
13. Gupta, V.; Fernandez-Crehuet, J.M.; Hanne, T.; Telesko, R. Requirements Engineering in Software Startups: A Systematic Mapping Study. *Appl. Sci.* **2020**, *10*, 6125. [CrossRef]
14. Zhang, S.; Hao, L.; Schulzrinne, H. Policy Enforcement for IoT: Complexities and Emerging Solutions. In Proceedings of the 2024 IEEE International Performance, Computing, and Communications Conference (IPCCC), Orlando, FL, USA, 22–24 November 2024; pp. 1–2. [CrossRef]

15. Aguilar-Calderón, J.A.; Tripp-Barba, C.; Zaldívar-Colado, A.; Aguilar-Calderón, P.A. Requirements Engineering for Internet of Things (IoT) Software Systems Development: A Systematic Mapping Study. *Appl. Sci.* **2022**, *12*, 7582. [CrossRef]
16. Atzori, L.; Iera, A.; Morabito, G. The internet of things: A survey. *Comput. Netw.* **2010**, *54*, 2787–2805. [CrossRef]
17. Hull, E.; Jackson, K.; Dick, J. *Requirements Engineering in the Solution Domain*; Springer: London, UK, 2005.
18. Nuseibeh, B.; Easterbrook, S. Requirements engineering: A roadmap. In Proceedings of the Conference on the Future of Software Engineering, Limerick, Ireland, 9 June 2000; pp. 35–46.
19. Yu, E. Modelling Strategic Relationships for Process Reengineering. Ph.D. Thesis, Computer Science Department, University of Toronto, Toronto, ON, Canada, 1995.
20. Aguilar, J.A.; Garrigós, I.; Mazón, J.N.; Trujillo, J. An MDA Approach for Goal-oriented Requirement Analysis in Web Engineering. *J. Univers. Comput. Sci.* **2010**, *16*, 2475–2494.
21. Bass, L.; Bergey, J.; Clements, P.; Merson, P.; Ozkaya, I.; Sangwan, R. *A Comparison of Requirements Specification Methods from a Software Architecture Perspective*; Technical Report; Software Engineering Institute, Carnegie-Mellon University: Pittsburgh, PA, USA, 2006.
22. Chrissis, M.B.; Konrad, M.; Shrum, S. *CMMI for Development: Guidelines for Process Integration and Product Improvement*; Pearson Education: London, UK, 2011.
23. Dalpiaz, F.; Franch, X.; Horkoff, J. Istar 2.0 language guide. *arXiv* **2016**, arXiv:1605.07767.
24. Anda, A.A. Modeling Adaptive Socio-Cyber-Physical Systems with Goals and SysML. In Proceedings of the 2018 IEEE 26th International Requirements Engineering Conference (RE), Banff, AB, Canada, 20–24 August 2018; pp. 442–447. [CrossRef]
25. Ahmad, M.; Gnaho, C.; Bruel, J.M.; Laleau, R. Towards a Requirements Engineering Approach for Capturing Uncertainty in Cyber-Physical Systems Environment. In *Proceedings of the New Trends in Model and Data Engineering, Marrakesh, Morocco, 24–26 October 2018*; Abdelwahed, E.H., Bellatreche, L., Benslimane, D., Golfarelli, M., Jean, S., Mery, D., Nakamatsu, K., Ordonez, C., Eds.; Springer: Cham, Switzerland, 2018; pp. 115–129.
26. Bouskela, D.; Falcone, A.; Garro, A.; Jardin, A.; Otter, M.; Thuy, N.; Tundis, A. Formal requirements modeling for cyber-physical systems engineering: An integrated solution based on FORM-L and Modelica. *Requir. Eng.* **2022**, *27*, 1–30. [CrossRef]
27. Loucopoulos, P.; Kavakli, E.; Mascolo, J. Requirements Engineering for Cyber Physical Production Systems: The e-CORE approach and its application. *Inf. Syst.* **2022**, *104*, 101677. [CrossRef]
28. Calvaresi, D.; Claudi, A.; Dragoni, A.F.; Yu, E.; Accattoli, D.; Sernani, P. A Goal-Oriented Requirements Engineering Approach for the Ambient Assisted Living Domain. In Proceedings of the 7th International Conference on PErvasive Technologies Related to Assistive Environments, Rhodes, Greece, 27–30 May 2014. [CrossRef]
29. Eterovic, T.; Kaljic, E.; Donko, D.; Salihbegovic, A.; Ribic, S. An Internet of Things visual domain specific modeling language based on UML. In Proceedings of the 2015 XXV International Conference on Information, Communication and Automation Technologies (ICAT), Sarajevo, Bosnia and Herzegovina, 29–31 October 2015; pp. 1–5. [CrossRef]
30. Santos, L.; Pereira, J.; Silva, E.; Batista, T.; Cavalcante, E.; Leite, J. Identifying Requirements for Architectural Modeling in Internet of Things Applications. In Proceedings of the 2019 IEEE International Conference on Software Architecture Companion (ICSA-C), Hamburg, Germany, 25–26 March 2019; pp. 19–26. [CrossRef]
31. Morkevicius, A.; Bisikirskiene, L.; Bleakley, G. Using a systems of systems modeling approach for developing Industrial Internet of Things applications. In Proceedings of the 2017 12th System of Systems Engineering Conference (SoSE), Shanghai, China, 15–18 July 2017; pp. 1–6. [CrossRef]
32. Bing, K.; Fu, L.; Zhuo, Y.; Yanlei, L. Design of an Internet of Things-based smart home system. In Proceedings of the 2011 2nd International Conference on Intelligent Control and Information Processing, Harbin, China, 25–28 July 2011; Volume 2, pp. 921–924. [CrossRef]
33. Zambonelli, F. Towards a General Software Engineering Methodology for the Internet of Things. *arXiv* **2016**, arXiv:1601.05569. [CrossRef]
34. Bernabé, J.; Bouloukakis, G.; Mehrotra, S.; Venkatasubramanian, N. Abstracting interactions with IoT devices towards a semantic vision of smart spaces. In Proceedings of the 6th ACM International Conference on Systems for Energy-Efficient Buildings, Cities, and Transportation (BuildSys), New York, NY, USA, 13–14 November 2019.
35. Aguilar-Calderón, J.A.; Tripp-Barba, C.; Zaldivar, A.; Aguilar-Calderón, P.A.; Aguilar-Calderon, P.A. Empirical Evaluation of GORA. Article: Enhancing Sustainable IoT Systems through a Goal-Oriented Requirements Analysis Framework. *Dataset* **2025**. [CrossRef]
36. Docker Inc. Docker: Empowering App Development for Developers. 2024. Available online: https://www.docker.com (accessed on 6 May 2025).
37. Eclipse Foundation. Eclipse Mosquitto: An Open Source MQTT Broker. 2024. Available online: https://mosquitto.org (accessed on 6 May 2025).
38. IoTIFY. IoTIFY: Cloud-Based IoT Simulation Platform. 2024. Available online: https://www.iotify.io (accessed on 6 May 2025).

39. IBM Emerging Technology. Node-RED: Low-Code Programming for Event-Driven Applications. 2024. Available online: https://nodered.org (accessed on 6 May 2025).
40. InfluxData. InfluxDB: Time Series Database. 2024. Available online: https://www.influxdata.com/products/influxdb-overview/ (accessed on 6 May 2025).
41. Grafana Labs. Grafana: The Open Observability Platform. 2024. Available online: https://grafana.com (accessed on 6 May 2025).
42. Silva, D.; Gonçalves, T.G.; da Rocha, A.R.C. A Requirements Engineering Process for IoT Systems. In Proceedings of the XVIII Brazilian Symposium on Software Quality, Fortaleza, Brazil, 28 October–1 November 2019; pp. 204–209. [CrossRef]
43. Silva, D.V.d.; Gonçalves, T.G.; Travassos, G.H. A Technology to Support the Building of Requirements Documents for IoT Software Systems. In Proceedings of the 19th Brazilian Symposium on Software Quality, São Luís, Brazil, 1–4 December 2000. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.